Basics

Inserting

```
public static Node insert(Node head, int index, int elem) {
    Node newNode = new Node(elem);

    // Insert at the beginning
    if (index == 0) {
        newNode.next = head;
        return newNode;
    }

    // Insert in middle or end
    Node current = head;
    for (int i = 1; i < index  && current != null; i++) {
        current = current.next;
    }

    newNode.next = current.next;
    current.next = newNode;

    return head;    // Can't return newNode
}
```

Deleting

```
 public static Node delete(Node head, int index) {

    // Delete the first node
    if (index == 0 && head!=null) {
        head= head.next;
    }

    // Delete from middle or end
    Node current = head;
    for (int i = 0; i < index - 1 && current != null; i++) {
        current = current.next;
    }

    current.next = current.next.next;
```

```
    return head;
}
```

Copying

```
Node head2= new Node(head.elem, null);
Node current2=head2;

Node current=head.next;

        while(current!=null){
            current2.next=new Node(current_main.elem);
            current2=current2.next;
            current=current.next;
        }
```

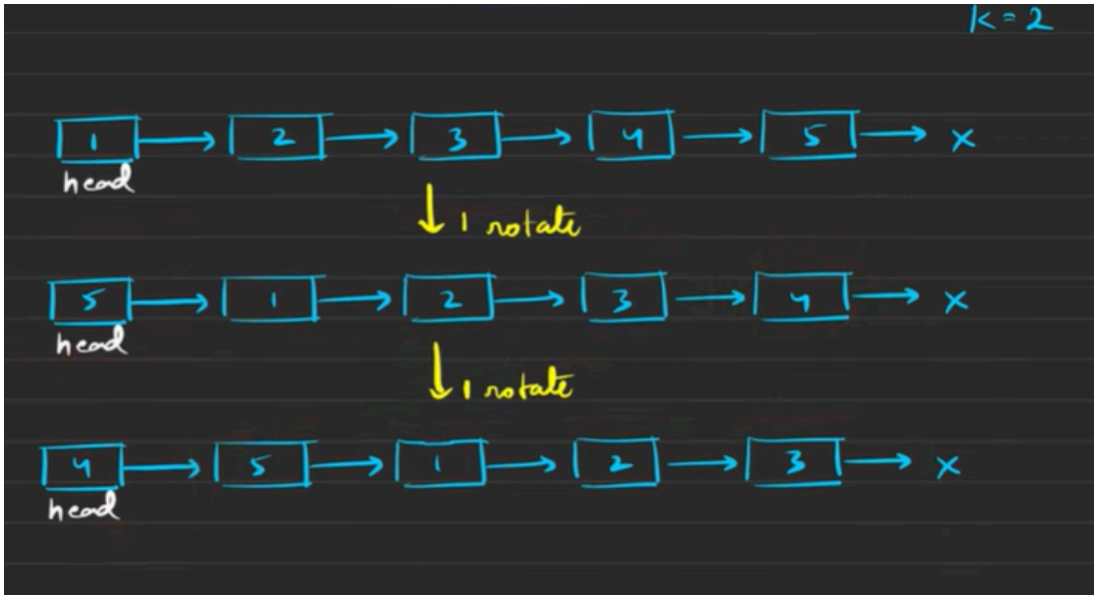
Reverse

```
  Node prev=null;
  Node current=dHead.next;     [Non dummy headed current = head]
  Node next=null;

        while(current!=null){
            next= current.next;
            current.next=prev;
            prev=current;
            current=next;
        }

        return prev;
    }
```

Array to LinkedList

```
public static Node createList( Object[] arr ){
    Node head = new Node( arr[0] );
    Node n = head;

    for ( int i=1; i<arr.length; i++ ){    //Array thakle for loop
        Node newN = new Node( arr[i] );
        n.next = newN;
        n = n.next;
    }
    return head;
}
```

Right Rotate




```
//Count length
Node current1= head;
 int length = 1;
 while (current1.next != null) {
    current1 = current1.next;
    length++;
 }
Node tail=current1;
```

```
    tail.next = head;
     k = k % length;
     int stepsToNewHead = length - k;

     // Move to the new tail
     Node newTail = head;
     for (int i = 1; i < stepsToNewHead; i++) {
         newTail = newTail.next;
     }

     Node newHead = newTail.next;   //Tail r por jeta chilo oita new head
     newTail.next = null;

     return newHead;
 }
```

Left Rotate

```
Node tail = head;
    int length = 1;
    while (tail.next != null) {
        tail = tail.next;
        length++;
    }

    tail.next = head;

    k = k % length;
    Node newTail = head;
    for (int i = 1; i < k; i++) {
        newTail = newTail.next;
    }

    Node newHead = newTail.next;
    newTail.next = null;

    return newHead;
```

```
}
```

Lab Task

Task 1

By now, you can quite understand the suitable data structure to represent a conga line. Now you are the choreographer of the Conga Dance in a Summer Festival. You wish to arrange the conga line **ascending** age wise and tell the participants to stand in a line likewise. Now as technical you are, can you write a method that will take the conga line and return True if everyone stands according to your instruction. Otherwise returns False.

Sample Input	Sample Returned Result
10 → 15 → 34 → 41 → 56 → 72	True
10 → 15 → 44 → 41 → 56 → 72	False

```
public static Boolean assembleCongaLine(Node head){

    Boolean congo=true;
    Node p=head;

    while(p.next!=null){            //p.next porjonto limit na dile IF condition e pera hobe
      if((int)p.elem>(int)p.next.elem){
          congo=false;
          break;
      }
      p=p.next;
    }
    return congo;
  }
```

Task 2

2. Word Decoder

Suppose, you have been hired as an cyber security expert in an organization. A mysterious code letter has been discovered by your team, your task is to decode the letter. After lots of research you found a pattern to solve the problem. Problem details are given below:

- For each encoded word a linked list will be given, where each node will carry one letter as an element.
- For the decoded word, the letters are at the multiples of $(13 \% length\ of\ linkedlist)$ position **[0 indexed].** For example, if the length of the given linked list is 10, then 13 % 10 = 3 and the letters are at positions which are multiples of 3 (i.e. at position 3, 6, 9)
- However, the letters are stored in the given linked list in opposite order. Thus the decoded word has to be reversed.
- After decoding, your program should return a dummy headed singly linked list containing the decoded word.

Sample Input	Sample Output
B→M→D→T→N→O→A→P→S→C	None → C→A→T **Explanation:** Length of the list= 10 & 13%10=3 The letters are T, A, C at 3, 6, 9 positions respectively **[0 indexed].** The letters are reversed and the resulting linked list is None → C→A→T
Z→O→T→N→X	None → N **Explanation:** Length of the list= 5 & 13%5=3. The letter is N at position 3 **[0 indexed].** The letters are reversed and the resulting linked list is None → N

```
public static Node wordDecoder( Node head ){

  //Counting Key
  Node current=head;
  int count=0;
  while(current!=null){
     count++;
     current=current.next;
  }
  int key= 13%count;


  //New linked list
```

```
    Node dHead= new Node(null, null);
    Node current2=dHead;

    Node current3=head;
    int count2=0;

    while(current3!=null){
        if(count2%key==0 && count2!=0){
            current2.next=new Node(current3.elem);
            current2=current2.next;
        }
        count2++;
        current3=current3.next;
    }


    //Reverse
    Node prev=null;
    Node current4=dHead.next;
    Node next=null;

    while(current4!=null){
        next= current4.next;
        current4.next=prev;
        prev=current4;
        current4=next;
    }

    return prev;            //Always head return hoy except when it's reversed
}
```

Lab Assignment

Sample Input	Sample Output
building_1 = Red→ Green→ Yellow→ Red→ Blue→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green	Similar
building_1 = Red→ Green→ Yellow→ Red→ Yellow→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green	Not Similar
building_1 = Red→ Green→ Yellow→ Red→ Blue→ Green building_2 = Red→ Green→ Yellow→ Red→ Blue→ Green→ Blue	Not Similar

```
//1
 public static String checkSimilar( Node building1, Node building2 ){

    Node current1=building1;
    Node current2=building2;

    boolean similar=false;
    String type="";

    //Checking
    while(current1!=null && current2!=null){
        String a=(String)current1.elem;
        String b=(String)current2.elem;

        if(a.equals(b)){
            similar=true;
        }
        else if(!a.equals(b)){
            similar=false;
            break;
       }
        current1=current1.next;
        current2=current2.next;
    }


    //Last e jawar por baki thakle
    if(current1!=null && current2==null){
        similar=false;
```

```
    }

    else if(current1==null && current2!=null){
        similar=false;
    }


    if(similar==true){      //Don't use "=" it means assign kora
        type="Similar";
    }
    else if(similar==false){
        type="Not Similar";
    }

    return type;
}
```

Sample Given LL and Array Example1	Output
Dune → IT → Coraline → Inferno → Twilight [8, 10, 5, 10, 6]	IT → Inferno → Dune → Twilight → Coraline
Explanation Example1: In the given list, Dune has a score of 8, IT has 10, Coraline has 5, Inferno has 10, and Twilight has 6. The list is reorganized so that books with higher popularity appear first. Therefore, IT and Inferno, both with a score of 10, come to the front. Next is Dune with a score of 8, followed by Twilight with 6, and finally Coraline with 5.	
Sample Given LL and Array Example2	**Output2**
Hamlet → Persuasion → It → Dracula → Beloved [7, 9, 9, 6, 7]	Persuasion → It → Hamlet → Beloved → Dracula
Sample Given LL and Array Example3	**Output3**
Matilda → Franny → Foundation → Carrie → Misery [5, 8, 8, 10, 6]	Carrie → Franny → Foundation → Misery → Matilda

//2

```
public static Node organizeBooks(Node head, Integer[] popularity) {

   for(int i=0; i<popularity.length-1; i++){
      Node current=head;  //Every time loop e current ke front e back korano lagbe

     .  for(int j=0; j<popularity.length-1-i; j++){
         if(popularity[j]<popularity[j+1] && current.next!=null){

         Object temp1=current.elem;          //Elements exchange, not the nodes
         current.elem=current.next.elem;
         current.next.elem=temp1;

         int temp2=popularity[j];
         popularity[j]=popularity[j+1];
         popularity[j+1]=temp2;
         }

         current=current.next;
      . }
   }


  return head;
}
```

---

3. Alternate Merge

You are given two singly non-dummy headed linked lists. Your task is to write a function **alternate_merge(head1, head2)** that takes the heads of the two linked lists and returns the head of a modified linked list with all the elements of the two lists in alternate order. It is guaranteed that alternate placement is always possible. Your resulting linked list will always start with the head of linked list 1.

Input	Output
List1: 1 → 2 → 6 → 8 → 11 → None List2: 5 → 7 → 3 → 9 → 4 → None	1 → 5 → 2 → 7 → 6 → 3 → 8 → 9 → 11 → 4 → None
List1: 5 → 3 → 2 → -4 → None List2: -4 → -6 → 1 → None	5 → -4 → 3 → -6 → 2 → 1 → -4 → None
List1: 4 → 2 → -2 → -4 → None List2: 8 → 6 → 5 → -3 → None	4 → 8 → 2 → 6 → -2 → 5 → -4 → -3 → None

NOTE: The space complexity of the solution must be O(1). Which means, You're NOT ALLOWED to create any new linked list for this task.

//3

```
public static Node alternateMerge( Node head1, Node head2 ){
    Node current1=head1;
    Node current2=head2;

    while(current1!=null && current2!=null){   //Ulta right angle triangle r moto traverse korbe
        Node next1=current1.next;
        Node next2=current2.next;

        current1.next=current2;
        current2.next=next1;

        current1=next1;
        current2=next2;

    }


    return head1;
}
```

4. ID Generator

Write a function **idGenerator()** that will take three non-dummy headed linear singly linked lists containing only digits from 0-9 and **generate another singly linked list** with 8 digit student ID from it [The student ID contains only digits from 0 to 9].

The first linked list will have the first four digits of the student id in reverse order. The remaining four digits can be obtained by adding the elements of the second linked list **from the beginning** with the elements of the third linked list **from the beginning**.

If the summation is >= 10, then you have to mod the summation by 10 and insert the result in the output linked list. For example, in sample input 2, the last element of the second list is 7 and the last element of the third linked list is 8. So after adding them we get 7+8=15>10. So the digit we will insert as an element of the Student ID list, will be 15%10=5.

Sample Input 1: 0 → 3 → 2 → 2 5 → 2 → 2 → 1 4 → 3 → 2 → 1 **Sample output 1:** 2 → 2 → 3 → 0 → 9 → 5 → 4 → 2	**Sample Input 2:** 0 → 3 → 9 → 1 3 → 6 → 5 → 7 2 → 4 → 3 → 8 **Sample output 2:** 1 → 9 → 3 → 0 → 5 → 0 → 8 → 5

//4

```
public static Node idGenerator(Node head1, Node head2, Node head3) {

    //Reversing first LL
    Node prev= null;
    Node n1=head1;
    Node next=null;

    while(n1!=null){        //n1.next dile last e reverse hobe na
        next=n1.next;
```

```
        n1.next=prev;
        prev= n1;
        n1=next;
    }


    //n1 ke last e back korano cuz n1 NULL e chole gese
    n1=prev;
    while(n1.next!=null){
        n1=n1.next;
    }



    //Making a new LL by adding both
    Node n2=head2;
    Node n3=head3;

    while(n2!=null && n3!=null){
        Node add=new Node(null, null);

        int sum=(int)n2.elem + (int)n3.elem;
        if(sum>=10){
            sum=sum%10;
        }

        add.elem=sum;
        n1.next=add;
        n1=n1.next;

        n2=n2.next;
        n3=n3.next;
    }

    return prev;

}
```

Lab Test

You are given the head of a singly linked list of integers. A node in this list is considered **BAD** if its value is strictly smaller than both its **previous** and **next** node values. Your task is to write a function ***moveTheBad*** to move all such **BAD** nodes to the tail of the list. Return the modified head. Determine the time complexity of your solution.

Note:

- The first and last nodes of the list are never considered bad, as they do not have both a previous and a next node.
- The list must be updated in-place (i.e., modify the list structure without using extra memory for a new list).
- Consider the previous and next node from the original list (not from the updated list)

No other data structures can be used other than linked lists.
Consider that node class is already provided.

Sample Input	Sample Output & Explanation
Linked List: 10 → 5 → 9 → 3 → 8 → 2 → 9	Linked List: 10 → 9 → 8 → 9 → **5** → **3** → **2** **Explanation:** Node with value 5 (smaller than 10 and 9) Node with value 3 (smaller than 9 and 8) Node with value 2 (smaller than 8 and 9)
Linked List: 8 → 4 → 7 → 6 → 2 → 12 → 9	Linked List: 8 → 7 → 6 → 12 → 9 → **4** → **2** **Explanation:** Node with value 4 (smaller than 8 and 7) Node with value 2 (smaller than 6 and 12)

```
public static Node moveTheBad(Node head) {
   if (head == null || head.next == null || head.next.next == null) return head;

   Node ogtail = head;
   while (ogtail.next != null) ogtail = ogtail.next;
   Node tail = ogtail;

   Node prev = head;
   Node current = head.next;
   Node next = current.next;
   Node stop = ogtail.next;

   while (next != stop) {
      if (current.val < prev.val && current.val < next.val) {
         prev.next = next;
         tail.next = current;
         tail = current;
         current.next = null;

         current = next;
```

```
            next = next.next;
        } else {
            prev = prev.next;
            current = current.next;
            next = next.next;
        }
    }
    return head;
}
```

A robotics company stores task priorities in a singly linked list, where each node's value represents the time required to complete the task. To balance workload, tasks should be rearranged in an alternating low-high pattern:

task1 < task2 > task3 < task4 > task5 …

This means the first task takes less time than the second, the second takes more time than the third, and so on.

NOTE: You cannot create any new linked lists or use any other data structures.

Sample Input	Sample Output
4 → 3 → 7 → 8 → 6 → 2 → 1	3 → 7 → 4 → 8 → 2 → 6 → 1 Explanation: 3 < 7 > 4 < 8 > 2 < 6 > 1
1 → 4 → 3 → 2 → 5	1 → 4 → 2 → 5 → 3 Explanation: 1 < 4 > 2 < 5 > 3

According to your programming language preference, complete the following methods only:

Java	Python

public Node rearrangeZigZag(Node head){
#To Do
}
	def rearrangeZigZag(head):
	//To Do

```
public static Node rearrangeZigZag(Node head) {
    if (head == null || head.next == null) return head;

    Node current = head;
    boolean less = true;        //To flip

    while (current != null && current.next != null) {
```

```
        if (less == true) {
            if (current.val > current.next.val) {   //flip r shathe match kore na(opposite change)
                int temp = current.val;
                current.val = current.next.val;
                current.next.val = temp;
            }
        }

        else if(less==false) {
            if (current.val < current.next.val) {     //flip r shathe match kore na
                int temp = current.val;
                current.val = current.next.val;
                current.next.val = temp;
            }
        }
        current = current.next;
        less = !less;                   // flip pattern
    }

    return head;
}
```

1. ***How can this be implemented without swapping node data?***
   Instead of swapping the values, we can rearrange the links between nodes. Keep track of the previous, current, and next nodes.

2. ***If all elements are equal, will the zig-zag order differ from the original list? Why?***
   No, it will not differ. Because the condition current.val < current.next.val or current.val > current.next.val will never be true when all elements are equal.

3. ***What will happen if the list is already in increasing order?***
   The algorithm reorders it into the required zig-zag pattern.

4. ***If the linked list has only one node, will the algorithm make any changes? Why or why not?***
   Because there is no next node to compare or swap with. The condition current != null && current.next != null fails immediately.

5. ***Why is it important to avoid using extra data structures in this problem?***
   Using extra structures (like arrays or new lists) increases space complexity to O(n).

6. ***Time complexity to count elements in a linked list- O(n)***

Question: You are given two singly linked lists A and B of the same length. Check whether the difference A[i] - B[i] at each corresponding node forms a strictly **increasing** sequence with a constant step size of 1.

Python:	**Java:**

def is_difference_linear_increasing(headA, headB):
# ToDo 	
pass
	public static boolean
	is_difference_linear_increasing(Node headA, Node headB) {
	// ToDo }

Return True if it does, otherwise False

Notes:

1. For Simplicity, you can assume the Linked List has an even number of elements
2. No need to write the Node class. Just assume Node class is there with two instance variables; ***one is elem,*** and the ***other is next***.

Sample Given LinkedList	**Sample Output 1**	**Explanation**
headA => 10 -> 12 -> 14 -> 16 **headB => 4 -> 5 -> 6 -> 7**	True	(10-4) = 6, (12-5) = 7, (14-6) = 8, (16-7) = 9 The differences are 6, 7, 8, 9, which are increasing by 1.
Sample Given LinkedList	**Sample Output 2**	**Explanation**
headA => 20 -> 24 -> 30 **headB => 10 -> 12 -> 15**	False	(20-10) = 10, (24-12) = 12, (30-15) = 15 The differences are 10. 12, 15, thus is not an increasing sequence of 1.

```
public static boolean is_difference_linear_increasing(Node headA, Node headB) {
    if (headA == null || headB == null) return false;


    int prevDiff = headA.elem - headB.elem;
    Node currentA = headA.next;
    Node currentB = headB.next;


    while (headA != null && headB != null) {
      int currDiff = currentA.elem - currentB.elem;

      if (currDiff != prevDiff + 1) {
        return false;
      }
```

```
            prevDiff = currDiff;
            currentA = currentA.next;
            currentB = currentB.next;
        }

        return true;
    }
```

You are given the head of three linked lists: **L1**, **L2**, and **L3**, all of the same length.

At each position, take the sum of the values from **L1** and **L2**, and also take the sum of the values from **L2** and **L3**. If at every position any one of these sums is odd and the other is even, return **True**. Otherwise, return **False**.

```
def sum_evenOdd(L1, L2, L3):
    # ToDo
    pass
```

Note: No need to write the Node class. Just assume Node class is there with two instance variables; ***one is elem*** and the ***other one is next***.

Sample Given LinkedList	Sample Output 1	Explanation
L1: [1 → 2 → 4 → 3] L2: [4 → 4 → 5 → 3] L3: [2 → 1 → 1 → 4]	True	At the first position: L1 + L2 = 1 + 4 = 5 (odd) L2 + L3 = 4 + 2 = 6 (even) At the second position: L1 + L2 = 2 + 4 = 6 (even) L2 + L3 = 4 + 1 = 5 (odd) At the third position: L1 + L2 = 4 + 5 = 9 (odd) L2 + L3 = 5 + 1 = 6 (even) At the fourth position: L1 + L2 = 3 + 3 = 6 (even) L2 + L3 = 3 + 4 = 7 (odd)
Sample Given LinkedList	**Sample Output 2**	**Explanation**
L1: [1 → 2 → 3] L2: [4 → 3 → 6] L3: [5 → 7 → 1]	False	At the first position: L1 + L2 = 1+4 = 5 (odd) L2 + L3 = 4+5 = 9 (odd) (both odd) → already fails..

Short Questions:

1. Determine the time complexity of your proposed solution.
2. In the task above, if at every position we take the sum of l1 and l3, and the result is an odd number at every position, can we draw any conclusion from that?
3. Will it be of any help if the given Linked Lists are Doubly Linked List? Explain Briefly.
4. Provide an example where using a dummy head node simplifies the implementation of a linked list operation.
5. Propose a scenario where an array provides better performance than a linked list.

```
boolean sum_evenOdd(Node L1, Node L2, Node L3) {
   while (L1 != null && L2 != null && L3 != null) {
      int s1 = L1.elem + L2.elem;
      int s2 = L2.elem + L3.elem;


      if ((s1 % 2) == (s2 % 2)) {
         return false;
```

```
        }

        L1 = L1.next;
        L2 = L2.next;
        L3 = L3.next;
    }

    return true;
}
```

1. Determine the time complexity of your solution.
Time Complexity: O(n)
Space Complexity: O(1)
(You're not creating extra data structures.)

2. If L1+L2 and L2+L3 both produce only odd sums at every position, what conclusion can we draw?
If both sums are always odd, it means L1 and L3 have the same parity at every index.
(Parity means both are either even-or-even or odd-or-odd.)

3. Would it help if the lists were doubly linked lists? Explain briefly.
No meaningful benefit.
You only move forward through the lists and never need to move backward or delete nodes.
So a doubly linked list adds memory and pointer overhead without improving performance.

Given a singly linked list, use the fast and slow pointer approach to:
Determine whether the list length is even or odd.

Return the middle node:

- If the length is odd → return the exact middle.
- If the length is even → return the second middle.

- List: 7 → 14 → 28 → 35 → 49 [Odd so output return 28]
- List: 5 → 11 → 19 → 24 → 32 → 48 [Even so output 19]

```
public static Node findMiddle(Node head) {
    Node slow = head;
    Node fast = head;

    while (fast != null && fast.next != null) {
        slow = slow.next;
        fast = fast.next.next;
    }

    return slow;
}
```

RFTS Practice

Suppose, you are working on a web application which has a customer support system. You are assigned to handle the customer support tickets. The support tickets are stored in a non-dummy headed singly linear linked list on a first-come first-served basis by IDs. However, due to a glitch, some tickets have been duplicated. Now, your task is to complete the given method **remove_Duplicates(head)** which takes the head of the linked list as input to remove the duplicate IDs ensuring that each ticket appears only once.

[DO NOT USE OTHER DATA STRUCTURE OTHER THAN GIVEN LINKED LIST]

Sample Input	Sample Output	Explanation
Input Tickets: 101 -> 103 -> 101 -> 102 -> 103 -> 104 -> 105 -> 105	Fixed Tickets: 101 -> 103 -> 102 -> 104 -> 105	All the duplicates of 101, 103 and 105 have been removed.
Input Tickets: 102 -> 101 -> 101 -> 102 -> 102 -> 102 -> 103 -> 104 -> 104	Fixed Tickets: 102 -> 101 -> 103 -> 104	All the duplicates of 102, 101 and 104 have been removed.

```
public static Node remove_Duplicates(Node head) {
   if (head == null) {
         return head;
         }

   Node current = head;
   while (current != null) {
   Node runner = current;

    while (runner.next != null) {
      if (runner.next.data == current.data) {
            runner.next = runner.next.next;
       } else {
             runner = runner.next;
      }
    }
      current = current.next;
   }

   return head;
}
```

You are given a non-dummy-headed, singly linear linked list containing positive integers. Your task is to complete the given method **reverseAndSwap()** that takes the **head** of the linked list and an **integer i** as input and returns the head of the modified list.

Your task is to

- Reverse the list from index 0 to i
- Swap the two parts of the list, i.e., the unchanged part from index i + 1 to total_nodes – 1 will come before the reversed part from index 0 to i in the new list. (where total_nodes = number of nodes in the linked list)

Check the given input and output for more clarification.

Note: You can assume that the index i will always be in the range 0 to total_nodes – 1. Assume Node class has elem and next variable. No need to write the Node class.

[DO NOT USE OTHER DATA STRUCTURE OTHER THAN GIVEN LINKED LIST, i.e. YOU CANNOT CREATE A NEW LINKED LIST]

Sample Input	Sample Output	Explanation
5→7→6→3→8→2→1 i = 3	8→2→1→3→6→7→5	The list is reversed from index 0 to 3 producing 3→6→7→5. The unchanged part (8→2→1) now sits before the reversed part, producing the output list.

```
public static Node reverseAndSwap(Node head, int i) {
    if (head == null || head.next == null){
        return head;
    }

    // Step 1: Reverse
    Node current = head;
    Node prev = null;
    Node next = null;
    int count = 0;

    while (current != null && count <= i) {
        next = current.next;
        current.next = prev;
        prev = current;
        current = next;
        count++;
    }

    // Step 2: find the end of unchanged part
    Node newHead = current;
    Node temp = current;
```

```
    while (temp != null && temp.next != null) {
        temp = temp.next;
    }


     temp.next = prev;

    return newHead;
}
```

You are given a non-dummy headed singly linear linked list containing positive integers. Your task is to complete the given method **IsSumPossible()** which takes the head of the linked list and an integer, n as input. If the integer n can be obtained by summing any two of the elements of the linked list, return True, else return False.

Hint: There might be more than one such pair possible, you don't have to check all such pairs.

[DO NOT USE OTHER DATA STRUCTURE OTHER THAN GIVEN LINKED LIST]

Sample Input	Returned Value	Explanation
list = 1→2→3→4→5 n = 7	True	Sum of the elements (3,4) or (2,5) equals 7
list = 1→2→4→5→6 n = 4	False	There are no two elements that make the sum 4. Note that though there is an element 4 itself in the list, the output will be False.
list = 5 n = 5	False	There is only one element so, the sum of two elements cannot be 5

```
    Node current1 = head;
    while (current1 != null) {
        Node current2 = current1.next;
        while (current2 != null) {
            if (current1.data + current2.data == n) {
                return true;
            }
            current2 = current2.next;
        }
        current1 = current1.next;
    }

    return false;
}
```

You are given two heads of a singly linked list that may or may not intersect at some node. If they do intersect, they share a common tail segment (i.e., from the intersecting node onward, both lists are identical in structure and content). Your task is to write a method called **mergeLL(h1, h2)** that takes the two heads as parameters and **returns the merged head**.

This method should **merge the second linked list into the first**, only if the lists intersect. The resulting merged list should be a single list that starts from h1 and includes all nodes, avoiding any duplication of the overlapping segment.

If there's an intersection, the merge should connect the non-overlapping portion of h1 to the head of h2, up to the point where both lists start sharing nodes. If there's no intersection, then the program will return None/null. The Node class has an elem variable (Integer type) and a next variable (Node type).

Restrictions:

- **Do not create any new nodes.**
- **Do not modify the values in the existing nodes.**
- **Do not create any instances of any other data structures (e.g, array, stack, etc.)**

Sample Given Linked List	Returned Linked List
An intersected linked list example	
(h1) *56 → 78 → 91* ↘ ***62 → 17 → 89 → 24*** ↗ (h2) ***43 → 33***	*56→78→91→**43→33→62→17→89→24***
Non-intersecting linked list example	
(h1) 56 → 78 → 91→17 → 89 → 24 (h2) 43 → 33 → 36 → 29	*None/null*

```
int len1 = getLength(head1);
int len2 = getLength(head2);

int diff = Math.abs(len1 - len2);

if (len1 > len2) {
   for (int i = 0; i < diff; i++) current1 = current1.next;     //Same jaygay ene tarpor compare
   } else {
    for (int i = 0; i < diff; i++) current2 = current2.next;
    }

       while (current1 != null && current2 != null) {
          if (current1.next.elem == current2.next.elem) {
                current1.next=head2;
                return head1;
          }
          current1 = current1.next;
          current2 = current2.next;
```

```
    }

    return null;
  }
```

---

ii) Suppose, you are in charge of collecting customer tokens in a line and have to store them in a singly linked list in a first-come, first-served order. Your company's policy is to serve senior citizens first, but you noticed they were standing towards the back, and you know their starting positions. Your task is to rearrange the tokens in a way so that senior citizens are moved to the front.
Implement the method named **rearrange_Tokens(head, seniorPos)**, which takes the head of the linked list and the starting position of the senior citizens. It rearranges senior citizens to the front and others to the back and returns the rearranged linked list's head. If the senior position is invalid, return the original list unchanged.

[DO NOT USE OTHER DATA STRUCTURE OTHER THAN GIVEN LINKED LIST]

Sample Input	Sample Output	Explanation
Tokens list: A3 → A9 → A4 → A2 → A7 → A8 → A1 Senior citizen position: 4	Rearranged Tokens list: A2 → A7 → A8 → A1 → A3 → A9 → A4	Here the senior citizens start from position 4, hence all the tokens from position 4 to the end of the list were brought to the front and others are behind that list.
Tokens list: A9 → A3 → A4 → A8 → A6 → A5 Senior citizen position: 5	Rearranged Tokens list: A6 → A5 → A9 → A3 → A4 → A8	Here the senior citizens start from position 5, hence all the tokens from position 5 to the end of the list were brought to the front and others are behind that list.

```
public static Node rearrange_Tokens(Node head, int seniorPos) {
    if (head == null || seniorPos <= 0) return head;

    Node temp = head;
    int count = 1;


    while (count < seniorPos - 1 && temp != null) {
       temp = temp.next;
       count++;
    }


    if (temp == null || temp.next == null) return head;
```

```
    // Split the list
    Node seniorHead = temp.next;
    temp.next = null;            // Break the link


    Node tail = seniorHead;
    while (tail.next != null) {
        tail = tail.next;
    }
    tail.next = head;


    return seniorHead;
}
```

Q1. You are managing a Mission Control Task Queue for a space exploration system. The queue is stored as a **dummy-headed singly linked list**, where each node after the dummy head represents a mission task ID waiting to be executed. During system optimization, engineers discovered that certain mission tasks perform better when executed in alternating priority pairs. Therefore, Mission Control issues a command: **"Swap every consecutive pair of mission tasks in the queue. If the last task has no partner, it should remain unchanged."**

Now, your task is to implement the function **reversePair( ),** which takes the head of the singly linked list as a parameter and performs pairwise reverse operations. ***Assume that the Node class is already provided with the instance variables elem (Object) and next (Node).***

NOTE: You can't create any new node. The Space Complexity of the solution must be O(1).

Sample Input	Sample Output	Function/Method Skeletons

DH → 1 → 2 → 3 → 4	DH→ 2 → 1 → 4 → 3	**Java Notation:**
		public void reversePair(Node head){
		// Write your code here
		}

DH → 5 → 9 → 3	DH → 9 → 5 → 3	**Python Notation:** def reversePair(head): # Write your code here

```
public void reversePair(Node head) {
   Node current = head;

   while (current != null &&
         current.next != null &&
         current.next.next != null) {

      Node first = current.next;
      Node second = first.next;

      first.next = second.next;
      second.next = first;
      current.next = second;

      current = first;
   }
}
```
